

Integrations guide — NEDS tool ecosystem

The CRM connects to two other NEDS tools — **nedsdrishti.in** (Drishti) and **socialmediadost.com** (Social Media Dost / SMDost). This guide explains what flows automatically between the three tools, what each team member sees, and what to do when something doesn't look right.

This guide is for **managers and admins**. Sales, support and accounts staff will encounter the effects of these integrations in their day-to-day work — the relevant parts are called out in their own guides.

The three-tool picture

...

CRM (source of truth)

Clients · Deals · Invoices · Payments · Tickets

▼ ▼

nedsdrishti.in socialmediadost.com

Service delivery Content production

Audits · SEO · Brief → AI content

Analytics · Posts → Approval → Posting

...

The **CRM** is where client relationships, money, and support live. Drishti is where the service delivery team runs audits, tracking and content scheduling. SMDost is where the content team creates AI-generated posts, gets client sign-off, and queues them for publishing.

Integration 1 — Client auto-provisioning when a deal is won

What it does: When a Deal in the CRM is moved to the **Won** stage, the client is automatically created in Drishti and SMDost within seconds. No manual re-entry needed.

What the team sees:

- Sales marks the deal Won → the CRM creates the client in both tools in the background.
- The client's CRM profile gains two internal IDs: **Drishti Client ID** and **SMDost Client ID** (visible in the client record's detail section).

- An **activity entry** appears on the client's CRM timeline confirming provisioning, e.g. **"Provisioned in Drishti (ID: drsh-xyz)"**.

If provisioning fails:

- Check the client's CRM activity feed — a failure note will appear there too.
 - Re-trigger by opening the Deal, moving it back to Negotiation, and then to Won again — the job is idempotent (it won't duplicate).
 - If repeated failures occur, check the server `.env` for `DRISHTI_API_URL` , `DRISHTI_SERVICE_KEY` , and `SMDOST_SERVICE_KEY` .
-

Integration 2 — Brief approved in SMDost → draft invoice in CRM

What it does: When the content team marks all content in a brief as

Approved in SMDost, the CRM automatically creates a **Draft Invoice** for that client and notifies the accounts team.

What accounts sees:

- A bell notification: `"SMDost brief approved for [Client] — draft invoice ready to price."`
- A draft invoice appears for the client in the CRM (Invoices tab on the client profile). The line item is pre-filled with the service description; the accounts team sets the amount and sends the invoice.

What the content team needs to know:

- Make sure the client's SMDost account has the correct **CRM Client ID** set (done automatically during provisioning). If it's blank, the webhook has no way to match the brief to a CRM client and the invoice won't be created.
-

Integration 3 — Approved SMDost content → Drishti for scheduling

What it does: When a brief is fully approved in SMDost, each content item (caption + media) is automatically pushed to Drishti as a **Scheduled Post**. The Drishti team then reviews and confirms before the posts go live.

What the Drishti team sees:

- New posts appear in the Drishti posts queue with status `**Scheduled / Pending Approval**`.
- Platform mapping: Instagram, Facebook, LinkedIn, Twitter/X, TikTok, and Google Business posts each land on the correct platform account.
- Default scheduled date is the **10th of the brief's month at 9 AM**. The content team can set a specific date per item in SMDost before approving.

Nothing in the CRM changes for this flow — it runs between SMDost and Drishti directly.

Integration 4 — Drishti activity → CRM client timeline

What it does: When a post is **approved, rejected, or published** in Drishti, that event is written to the CRM client's activity feed.

What the team sees:

- Open any client profile → **Activity** tab → entries like:

"Drishti: post approved — 'Diwali offer caption'"

"Drishti: post published on Instagram"

- This means the CRM has a full picture of the client relationship — sales history, support history, and service delivery milestones — without anyone copy-pasting between tools.

Note: only clients with a Drishti Client ID will receive these events. The ID is set automatically when a deal is won (Integration 1).

Integration 5 — Monthly brief auto-creation

What it does: On the **1st of every month at 7:30 AM**, the CRM automatically creates a content brief in SMDost for each active project that has a Social Media or GMB service and a linked SMDost client.

Platform defaults per service:

Service	Platforms created
Social Media	Instagram (4 posts), Facebook (4 posts)
GMB	Google Business (4 posts)

What the content team sees:

- A new brief appears in SMDost on the 1st, ready to start producing content.

No one has to remember to create it.

- The brief title follows the format: **"[Client Name] — [Service] — [Month Year]"**.

Idempotency: if the command runs twice for the same month (e.g. a server restart), it won't create duplicate briefs — it checks the CRM activity log first.

Manual trigger (backfill):

```
```bash
php artisan app:create-monthly-briefs --month=2026-07
```
```

Run this on the server via SSH if a month was missed.

Integration 6 — Client portal single sign-on (SSO)

What it does: Clients who log into the NEDS CRM portal see a **"Your NEDS Tools"** section on their dashboard with buttons to open Drishti and/or SMDost — and they land already signed in, without a separate password.

Conditions for the button to appear:

- The client's CRM profile must have a **Drishti Client ID** (auto-set on deal

won) for the Drishti button.

- The client's CRM profile must have an **SMDost Client ID** for the SMDost button.

- The client must have a user account in the respective tool (created automatically on deal won).

How it works for the client:

1. Log into the CRM portal.
2. Click "**Open Drishti Dashboard**" or "**Open Social Media Dost**".
3. They are redirected and automatically signed into that tool.
4. The sign-in link expires after **10 minutes** — if the client reports a sign-in error, they should return to the portal and click the button again.

If a client can't sign in via SSO:

- Check that their CRM contact has a portal login (Clients → Contacts → invite if not).
- Check that the client's CRM profile has the correct Drishti/SMDost Client ID.
- Check that the client's user account exists in Drishti and is **active**.
- If issues persist, the client can log into Drishti or SMDost directly with their email and password.

Integration 7 — Drishti context link on support tickets

What it does: When a support ticket is about a client who is connected to Drishti, the ticket show page displays a blue "**Open in Drishti** →" link so the support agent can jump straight to the relevant Drishti view without searching.

The link is service-aware:

| Ticket service | Drishti link goes to |
|---------------------------------------|--------------------------------------|
| SEO or GMB | Client's audit list |
| Social Media or Performance Marketing | Client's optimization / content view |
| Any other / no service set | Client's detail page |

WhatsApp tickets: when a WhatsApp message creates a ticket and the client is linked to Drishti, the Drishti URL is also appended to the ticket description — so it's visible in wadesk.in as well.

Integration 8 — WhatsApp two-way reply (wadesk.in)

What it does: WhatsApp is a full two-way channel through the CRM, across **two separate numbers** run on wadesk.in — a **Support** line (post-sale, existing clients) and a **Marketing** line (pre-sale enquiries). Which line a message arrives on decides how the CRM routes it; wadesk.in tells the CRM which line via the `whatsapp_number` field on its webhook call.

- **Inbound, Support line:** wadesk.in calls the CRM webhook, which matches the phone number to a client and opens a **Ticket** (channel = WhatsApp) — deduplicated per wadesk.in conversation, so replies in the same conversation don't create new tickets. If the phone doesn't match any client, a **Lead** is created instead (source = WhatsApp), same as before.

- **Inbound, Marketing line: always** creates or updates a **Lead** — never a Ticket, even when the phone number matches an existing client. This is a deliberate routing rule (owner-confirmed 2026-08-03): the marketing number is pre-sale by definition, so an existing client messaging it by mistake still surfaces where Sales/Telecaller will see it.

- **Deduplication (both lines):** one Ticket or Lead per wadesk.in conversation — later messages in the same conversation are added as replies/notes rather than creating a duplicate. A new Lead is auto-assigned to a sales rep the same way any other new lead is (see the AI features section).

- **Full conversation capture, both directions:** wadesk.in notifies the CRM of **every** message in a conversation, not just its opening one — this includes a later reply from the customer, a staffer replying directly from wadesk.in's own inbox (not just from the CRM), and wadesk.in's AI after-hours assistant. On a Ticket this appears as a normal reply in the thread, correctly attributed ("Client" for the customer, the actual staffer's name, or "AI Assistant (WhatsApp)"), and a customer messaging again on a Resolved/Closed ticket reopens it automatically. On a Lead it appears as a note, prefixed `[Sent via WhatsApp by ...]` for an outbound message so it's clear who said what. A reply the CRM itself sent is never echoed back as a duplicate — wadesk.in knows not to re-notify its own service-key sends.

- **Outbound, Tickets:** when a staff member replies on a WhatsApp ticket in the CRM (and the reply is **not** marked "internal note"), the CRM sends it back through wadesk.in on the Support line automatically.

- **Outbound, Leads:** Sales/Telecaller/Admin/Manager can reply to a lead over WhatsApp too — the note box on a lead's page has an opt-in *****Also send as WhatsApp reply***** checkbox (unlike Ticket replies, a lead note is internal-only by default; you have to explicitly choose to send it). Only shown when the lead actually has an open WhatsApp conversation to reply on. A sent note gets a green "Sent via WhatsApp" badge in the timeline.

- **Deal-Won handoff:** the moment a deal moves to **Won**, the CRM automatically sends the new client an approved WhatsApp template on the **Support** line — the actual handoff from the marketing conversation to the support one. Needs `WADESK_HANDOFF_TEMPLATE_NAME` set in `.env` and a matching template approved both in Meta Business Manager and on

wadesk.in's Templates page; silently skipped (logged) until both exist.

- **Visibility Audit payment confirmation:** the moment someone pays for a Visibility Audit offer tier (`/offers/visibility-audit`), the CRM sends them an approved WhatsApp template on the **Marketing** line (`WADESK_MARKETING_NUMBER`) confirming the payment — same approve-then-set-env-var contract as the handoff message above, gated by `WADESK_VISIBILITY_AUDIT_TEMPLATE_NAME` .

What the team sees:

- A ticket tagged WhatsApp behaves like any other ticket — reply in the CRM as normal. Internal notes (the "internal" checkbox) are never sent to the customer — use these for team-only context.
- If a client can't be matched by phone number on the Support line, a **Lead** is created instead of a ticket (see above) — check the ****Lead Generation**** list rather than assuming the message was lost. If it's actually an existing client with an out-of-date phone number, fix the number on their client record and ask them to send another WhatsApp message so future replies open a proper ticket.
- Any message on the Marketing line always lands in **Lead Generation**, regardless of who the number belongs to — this is expected, not a bug.

If a customer says they didn't receive a reply:

- Ticket reply: check it wasn't marked as an internal note by mistake.
- Lead reply: check the "Also send as WhatsApp reply" checkbox was actually ticked — unlike Ticket replies, a lead note doesn't send by default.
- Check server `.env` has `WADESK_API_URL` and `WADESK_SERVICE_KEY` set — without these the outbound send is silently skipped (logged as a warning, never blocks the reply itself).
- wadesk.in outages never break the CRM reply — the message is just not forwarded; the staffer may need to resend once wadesk.in is back up.
- Who can actually see/reply to a conversation on wadesk.in itself (as opposed to whether the CRM sent it) is controlled entirely on wadesk.in's own Agents page, not here — see Section 13 of the admin guide.

Integration 9 — Drishti marketing metrics feed the monthly wins note

What it does: the CRM's AI monthly wins note (see the AI features section in the Admin/Manager guides) pulls real marketing-delivery numbers from Drishti for clients Drishti manages — posts published, audits completed, and marketing action items done for the month, fetched from Drishti's `GET /api/clients/{id}/monthly-metrics` endpoint via the same X-Service-Key pattern as the other integrations. These are the same counts Drishti's own weekly client digest already computes, just totalled over a full month

instead of a week.

What the team sees: no extra step — the monthly wins note simply mentions these numbers alongside the CRM's own (tasks/tickets/payments) when a client has a Drishti account linked. Nothing changes for clients without one.

If Drishti is unreachable: the call fails silently (logged as a warning) and the note still drafts from whatever the CRM itself knows. This integration can never block the monthly wins note from running.

Integration 10 — Meta Lead Ads webhook

Status: built, not yet configured — needs a Facebook Developer App, a registered Page, and a Page access token before it goes live. See Section 16 of the Admin guide for setup steps.

What it does: when someone submits a Facebook or Instagram Lead Ads form, Meta calls `POST /api/webhooks/meta-leads` — but that event only carries a `leadgen_id`, not the actual name/email/phone submitted. The CRM queues a job that fetches the real field data from Meta's Graph API

(`GET /{leadgen_id}?access_token=...`) and creates a Lead from it (source

Meta Ads). Deduped on `leads.meta_leadgen_id` — a redelivered webhook event never creates a second lead.

Auth (two different mechanisms, both required):

- `GET /api/webhooks/meta-leads` — Meta's one-time (and periodic) verification handshake. Authenticated by `hub.verify_token` matching `META_WEBHOOK_VERIFY_TOKEN` — no signature involved for this request.
- `POST /api/webhooks/meta-leads` — every real lead event. Authenticated by `X-Hub-Signature-256` (HMAC-SHA256 of the raw body, keyed with `META_APP_SECRET`), verified by `VerifyMetaWebhookSignature` middleware.

Field mapping: Meta's standard field names (`full_name` or `first_name` + `last_name`, `email` / `work_email`, `phone_number` / `work_phone_number`, `company_name`) map to the lead's core fields. Two custom questions are opportunistically mapped too, so a Meta lead scores as well as a manually-entered one: an answer that exactly matches an active **Service** name (case-insensitive) sets the lead's service, and an answer to a question whose text contains the word "**budget**" — with a parseable number or range (e.g. "₹10,000–25,000") — sets the estimated value. Any other custom question is preserved as a note on the lead instead. See

[Admin guide → Setting up Meta Lead Ads](#)

for how to word the ad form's questions to hit these.

`utm_source` / `utm_medium` are set to a fixed `meta` / `paid_social`;

`utm_campaign` stores the **ad's real name** as set in Ads Manager (fetched via one extra Graph API call, `GET /{ad_id}?fields=name`), falling back to

the lead form's name if there's no `ad_id`, and finally to the raw

`ad_id` / `form_id` if both name lookups fail — so the Lead Source Performance report's campaign breakdown reads like "SEO - Pune - July V2", not a numeric ID. That extra lookup is best-effort and never blocks lead creation: a failed or rate-limited call just falls back one step, silently.

What the team sees: no extra step once configured — the lead appears in

Lead Generation with source **Meta Ads**, auto-assigned and AI-scored like any other lead (Phase A applies automatically).

If the Graph API call fails (expired token, deleted lead, rate limit):

logged as a warning, no lead is created, and the job retries up to 3 times (60s backoff) before giving up silently — this must never break the webhook endpoint itself, which always responds 200 immediately after queueing.

Integration 11 — Telegram lead alerts

What it does: every new lead (any source) posts a short alert — name, source, assigned rep (or "unassigned"), and a link back into the CRM — to one shared Telegram group via the Telegram Bot API. A second, always-on place for the team to notice a new lead land, alongside the in-app notification bell and the auto-assigned rep's own notification.

Setup: message **@BotFather** on Telegram, create a bot, and set the token it gives you as `TELEGRAM_BOT_TOKEN`. Add the bot to the team's Telegram group, send any message in that group, then open

`https://api.telegram.org/bot<token>/getUpdates` and copy the negative number under `"chat":{"id": ... }` into `TELEGRAM_CHAT_ID`. See Section 13a of the admin guide.

If alerts stop arriving: check both env vars are set (silently skipped, logged as a warning, if either is missing — this must never block lead creation) and that the bot hasn't been removed from the group.

Integration 12 — Visibility Audit funnel tracking + WhatsApp recovery nudges

What it does: the `/offers/visibility-audit` page and its checkout link are tracking redirects, not raw links — every hit logs which funnel stage a lead reached (landing page viewed, checkout/Payment Page viewed) so the CRM can tell a lead who never engaged apart from a lead who got close and dropped off. Sales/Telecaller can review the stuck ones any time on

Lead Generation → Audit Recovery (see the Sales guide). On top of that, a scheduled job automatically nudges a stuck lead over WhatsApp so recovery doesn't depend on staff noticing during office hours: ``app:send-visibility-audit-recovery-nudges`` runs every 30 minutes and, for any lead

stuck at checkout 2+ hours or at the landing page 4+ hours with no nudge sent yet, sends an approved WhatsApp template on the **Marketing** line — `WADESK_VISIBILITY_AUDIT_RECOVERY_CHECKOUT_TEMPLATE_NAME` / `WADESK_VISIBILITY_AUDIT_RECOVERY_LANDING_TEMPLATE_NAME` , same approve-then-set-env-var contract as the handoff/payment-confirmation messages above. Each template's own CTA button (a WhatsApp "Dynamic URL" button, not a plain link in the message text) carries the lead back to the right funnel stage; a required "Stop promotions" button lets the lead opt out, since Meta classifies this as Marketing content, not a Utility message about an existing order.

A lead is only ever nudged once per stuck event — if they come back and drop off again later, that's a fresh nudge opportunity, but a single stall never sends the same message twice.

If nudges stop arriving: confirm both template names are set in `.env` (each stage silently no-ops on its own until its var is set) and that both templates are still Approved in Meta Business Manager **and** on wadesk.in's own Templates page — wadesk.in keeps its own copy of each template's approval status, which doesn't update automatically just because Meta approved it. Use wadesk.in's **Templates** → **Sync from Meta** button (Admin only) to pull the latest name/body/category/approval status straight from Meta instead of re-entering it by hand.

Checking integration health

All integration events leave a trace in the CRM:

| Where to look | What it shows |
|--|--|
| Client profile → Activity tab | Provisioning success/failure, Drishti events, brief creation |
| Client profile → Invoices tab | Draft invoices created by SMDost brief approvals |
| Tickets list → WhatsApp badge | Tickets auto-created from Support-line WhatsApp conversations |
| Drishti → Posts queue | Content pushed from SMDost |
| Ticket → replies | Outbound Support-line WhatsApp replies sent via wadesk.in |
| Lead Generation → source filter | Leads auto-created from Website, WhatsApp (both lines), and Meta Ads |
| Lead → notes → green "Sent via WhatsApp" badge | Outbound Marketing-line WhatsApp replies sent from a lead |
| Lead Generation → Audit Recovery | Leads currently stuck at the Visibility Audit landing page or checkout |

If any integration stops working, the most common causes are:

1. **Server** `.env` **out of date** — a key (`DRISHTI_SERVICE_KEY` , `SMDOST_SERVICE_KEY` , `PORTAL_SSO_SECRET` , `WADESK_API_URL` , `WADESK_SERVICE_KEY` , `META_APP_SECRET` , `META_WEBHOOK_VERIFY_TOKEN` ,

`META_PAGE_ACCESS_TOKEN` , `TELEGRAM_BOT_TOKEN` , `TELEGRAM_CHAT_ID` , etc.)

is missing or wrong.

Run `php artisan config:cache` after any `.env` change.

1. **Docker not restarted after env change on VPS** — use

`docker compose up -d` (not `restart`) so the container picks up new env vars.

1. **Client has no external ID** — if the deal was won before integrations were live, the client won't have a Drishti/SMDost ID. Provision manually by going to the deal, setting it to Won again (if safe) or by asking your developer to run the provisioning job directly.